

Covariant Return Types in Java

Last Updated : 03 Dec, 2021



As the ear hit eardrums “overriding” we quickly get to know that it can be done either virtue of different datatypes or arguments passed to a function what a programmer learned initially while learning [polymorphism in java](#). Before JDK 5.0, it was not possible to [override](#) a method by changing the return type. When we override a parent class method, the name, argument types, and return type of the overriding method in child class has to be exactly the same as that of the parent class method. The overriding method was said to be **invariant** with respect to return type.

Java version 5.0 onwards it is possible to have different return types for an overriding method in the child class, but the child’s return type should be a subtype of the parent’s return type. The overriding method becomes **variant** with respect to return type.

The co-variant return type is based on the [Liskov substitution principle](#).

Now geeks you must be wondering about why to use for which we will be listing down the advantages as follows:

- It helps to avoid confusing type casts present in the class hierarchy and thus making the code readable, usable and maintainable.
- We get the liberty to have more specific return types when overriding methods.
- Help in preventing run-time ClassCastExceptions on returns

Note: If we swap return types of Base and Derived, then above program would not work. Please see [this](#) program for example.

Example Two classes used for return types

```
// Java Program to Demonstrate Different Return Types
// if Return Type in Overridden method is Sub-type
```

```

// Class 1
class A {
}

// Class 2
class B extends A {
}

// Class 3
// Helper class (Base class)
class Base {

    // Method of this class of class1 return type
    A fun()
    {
        // Display message only
        System.out.println("Base fun()");

        return new A();
    }
}

// Class 4
// Helper class extending above class
class Derived extends Base {

    // Method of this class of class1 return type
    B fun()
    {
        // Display message only
        System.out.println("Derived fun()");

        return new B();
    }
}

// Class 5
// Main class
public class GFG {

    // Main driver method
    public static void main(String args[])
    {

        // Creating object of class3 type
        Base base = new Base();
    }
}

```

```
// Calling method fun() over this object
// inside main() method
base.fun();

// Creating object of class4 type
Derived derived = new Derived();

// Again calling method fun() over this object
// inside main() method
derived.fun();
    }
}
```

Output:

```
Base fun()
Derived fun()
```

 Comment

More info 

Next Article 

[How to Return an Array in Java?](#)

Similar Reads

Covariant Return Types in Java

As the ear hit eardrums "overriding" we quickly get to know that it can be done either virtue of different datatypes or arguments passed to a function what a programmer learned initially while learning...

 3 min read

How to Return an Array in Java?

An array is a data structure that consists of a group of elements of the same data type such that each element of the array can be identified by a single array index or key. The elements of the array are store...

 5 min read

Runtime Type Identification in Java

Runtime Type Identification in Java can be defined as determining the type of an object at runtime. It is extremely essential to determine the type for a method that accepts the parameter of type java.lang....

🕒 2 min read

Java System.exit(0) vs C++ return 0

Java and C++ are languages with different applications and design goals. C++ is an extension of procedural programming language C and Java relies on a Java virtual machine to be secure and highly...

🕒 3 min read

Type Erasure in Java

Prerequisite : Generics Generics concept is introduced in Java language to provide tighter type checks at compile time and to support generic programming. The way to implement generics, the Java compiler...

🕒 3 min read

Collections.nCopies() in Java

The role of Collections.nCopies() is to return an immutable list which contains n copies of given object. This function helps if we want to create a list with n copies of given object. The newly allocated data...

🕒 2 min read

Function Overloading and Return Type in C++

Function overloading is possible in C++ and Java but only if the functions must differ from each other by the types and the number of arguments in the argument list. However, functions can not be overloaded i...

🕒 2 min read

Java - Covariant Method Overriding with Examples

The covariant method overriding approach, implemented in Java 5, helps to remove the client-side typecasting by enabling you to return a subtype of the overridden method's actual return type. Covarian...

🕒 4 min read

Java Data Types

Java is statically typed and also a strongly typed language because, in Java, each type of data (such as integer, character, hexadecimal, packed decimal, and so forth) is predefined as part of the programming...

🕒 11 min read

Type conversion in Java with Examples

Java provides various data types just like any other dynamic languages such as boolean, char, int, unsigned int, signed int, float, double, long, etc in total providing 7 types where every datatype acquires...

🕒 6 min read

Pair Class in JavaTuples

A Pair is a Tuple from JavaTuples library that deals with 2 elements. Since this Pair is a generic class, it can hold any type of value in it. Since Pair is a Tuple, hence it also has all the characteristics of JavaTuple...

🕒 5 min read

Array get() Method in Java

The `java.lang.reflect.Array.get()` is an inbuilt method in Java and is used to return the element at a given index from the specified Array. Syntax `Array.get(Object []array, int index)` Parameters : This method...

🕒 3 min read

Array getInt() Method in Java

The `java.lang.reflect.Array.getInt()` is an inbuilt method in Java and is used to return an element at the given index from the specified Array as a int. Syntax `Array.getInt(Object []array, int index)` Parameters:...

🕒 3 min read

Automatic Type Promotion in Overloading in Java

Before going into the actual topic, first, we need to know about method overloading and type promotions. What is Method Overloading? When a class consists of more than one method with the same name but...

🕒 6 min read

Restrictions on Generics in Java

In Java, Generics are the parameterized types that allow us to create classes, interfaces, and methods in which the type of data they are dealing with will be specified as a parameter. This ensures the type...

🕒 5 min read

How compare() method works in Java

Prerequisite: Comparator Interface in Java, TreeSet in Java The `compare()` method in Java compares two class specific objects (x, y) given as parameters. It returns the value: 0: if (x==y) -1: if (x < y) 1: if (x >...

🕒 3 min read

Rules of Downcasting Objects in Java

Typecasting is one of the most important concepts which basically deals with the conversion of one data type to another datatype implicitly or explicitly. In this article, the concept of downcasting for objects is...

🕒 5 min read

CopyOnWriteArrayList toArray() method in Java

The `toArray()` method of `CopyOnWriteArrayList` is used to return an array containing all the elements in `CopyOnWriteArrayList` in the correct order. Syntax: `public Object[] toArray()` or `public <T> T[] toArray(T...`

Returning Multiple values in Java

Java doesn't support multi-value returns. We can use following solutions to return multiple values. If all returned elements are of same type We can return an array in Java. Below is a Java program to...

Article Tags :

Java

Practice Tags :

Java

